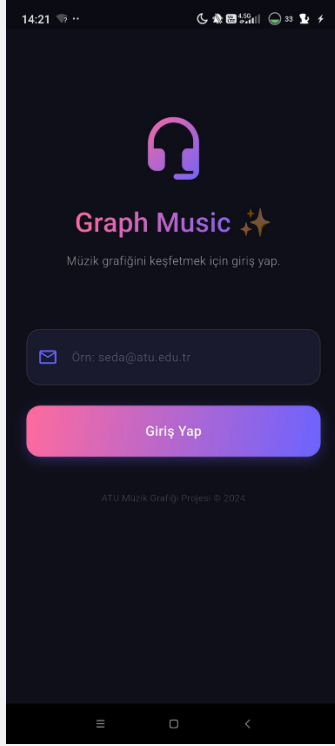


MusicGraph

A Social Music Discovery App Powered by Neo4j Graph Database



Application Home Screen — Welcome / Login Page

Term Project Report

May 2025

Adana Alparslan Türkeş University of Science and Technology

Department of Software Engineering

Seda Nur Öz-210129036

1. Project Overview

1.1 Project Description

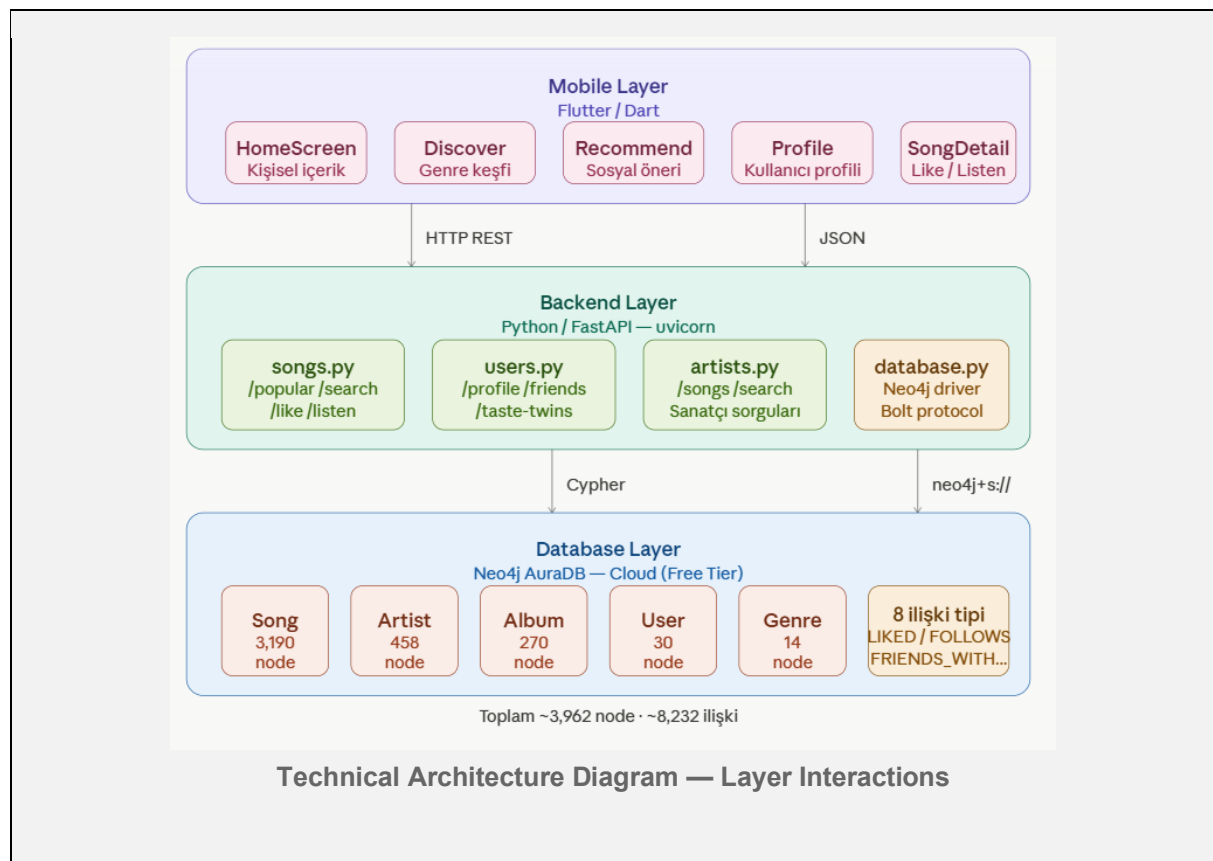
MusicGraph is a mobile music discovery and recommendation application built on top of a Neo4j graph database infrastructure, enriched with social networking features. The project was developed as a term project for the SEN-482 Special Topics in Software Engineering course.

The primary objective of the application is to demonstrate the advantages that graph databases offer over relational data models in social music recommendation scenarios. Friendship connections between users, song likes, and listening habits are leveraged through the natural traversal capabilities of graph structures to generate rich and meaningful recommendations.

1.2 Technical Architecture

The project consists of three main layers:

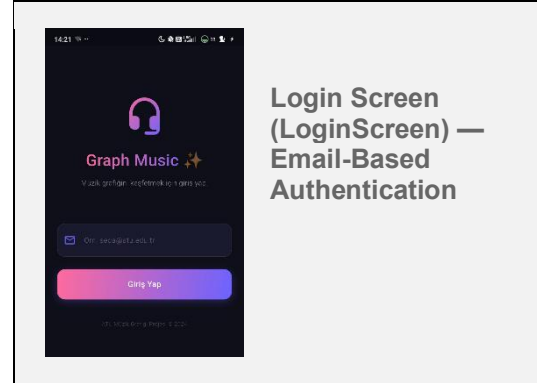
Layer	Technology	Responsibility
Database	Neo4j AuraDB (Cloud)	Graph data model, Cypher queries
Backend API	Python / FastAPI	REST endpoints, Neo4j driver integration
Mobile App	Flutter / Dart	User interface, API consumption



2. Application Features and Screens

2.1 User Login

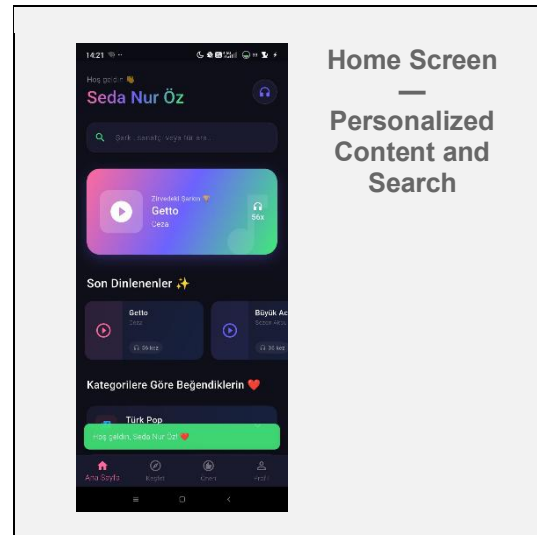
The application includes a login system based on SharedPreferences session management. Users authenticate via their email address and session data is stored on the device.



2.2 Home Screen (HomeScreen)

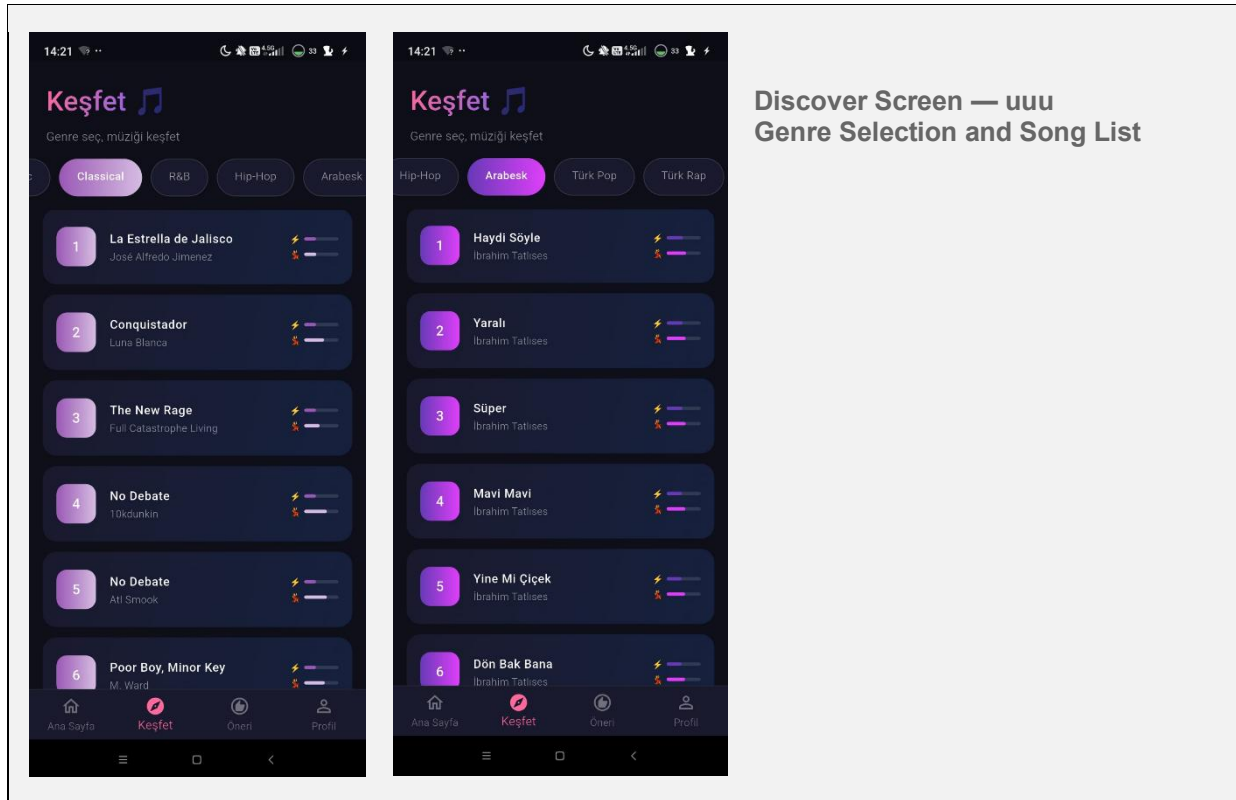
The home screen presents personalized content to the logged-in user:

- Top Song Card: The most-listened song is highlighted in a large hero card.
- Recently Played: Songs ordered by the count value in the LISTENED_TO relationship are shown as a horizontal list.
- Categorized Likes: Songs connected via the LIKED relationship are grouped by genre through BELONGS_TO and presented as collapsible expansion tiles.
- Search: Real-time search by song title or artist name.



2.3 Discover Screen (DiscoverScreen)

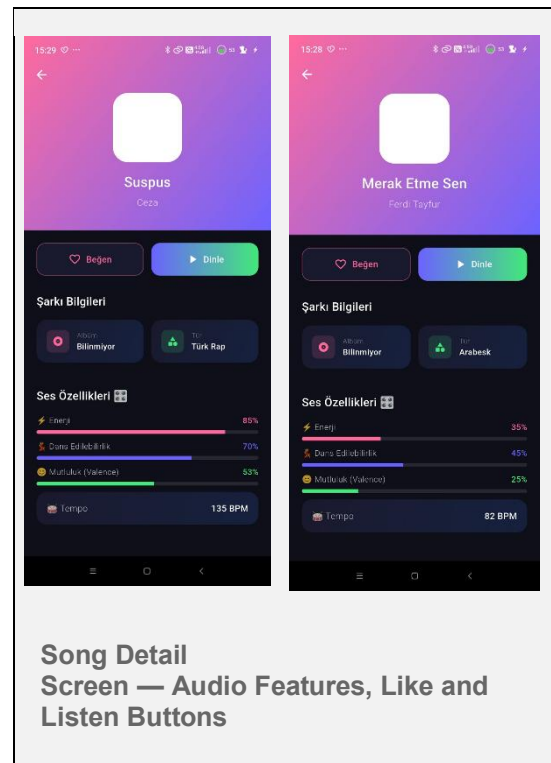
A genre-based music discovery screen. Users can select from 10 different music genres — Pop, Rock, Arabesk, Turkish Pop, Turkish Rap and more — to browse the highest-energy songs in that genre. Each song card visually displays energy and danceability bars.



2.4 Song Detail Screen (SongDetailScreen)

Tapping any song opens a detail screen containing:

- Song name, artist, album, genre, and release year
- Visual progress bar representation of energy, danceability, and valence values
- Tempo (BPM) value
- Like / Unlike button: Adds or removes the LIKED relationship in Neo4j in real time
- Listen button: Increments the count property of the LISTENED_TO relationship in Neo4j



2.5 Recommendations Screen (RecommendationsScreen)

This is the screen that best showcases the power of the graph database. It consists of four sections:

A. My Friends

Users connected through FRIENDS_WITH relationships are shown as a horizontal card list. Tapping a card navigates to that user's profile page.

B. Taste Twins

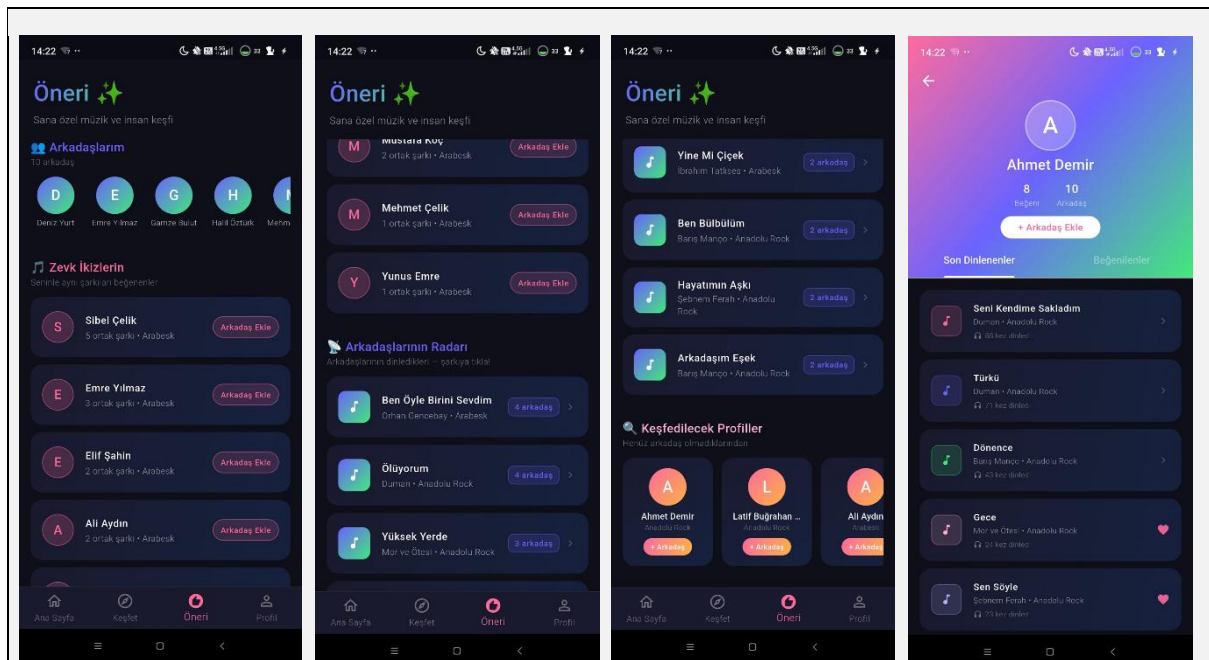
Users who share LIKED songs with the current user but are not yet friends are listed. This section embodies the core of graph traversal logic: User_A → [LIKED] → Song ← [LIKED] ← User_B. The number of common likes and top genre are shown, with a direct friend-add button.

C. Friends' Radar

Songs that the user's friends have listened to but the user has not yet heard are listed. Tapping a song opens a bottom sheet showing which friends liked or listened to that song.

D. Discover Profiles

Users who are not yet friends, ranked by most likes, are presented in a horizontal card list. Friends can be added directly or profiles can be visited from each card.



RecommendationsScreen —Friends, Taste Twins, Radar and Profile Discovery

2.6 User Profile Screen (UserProfileScreen)

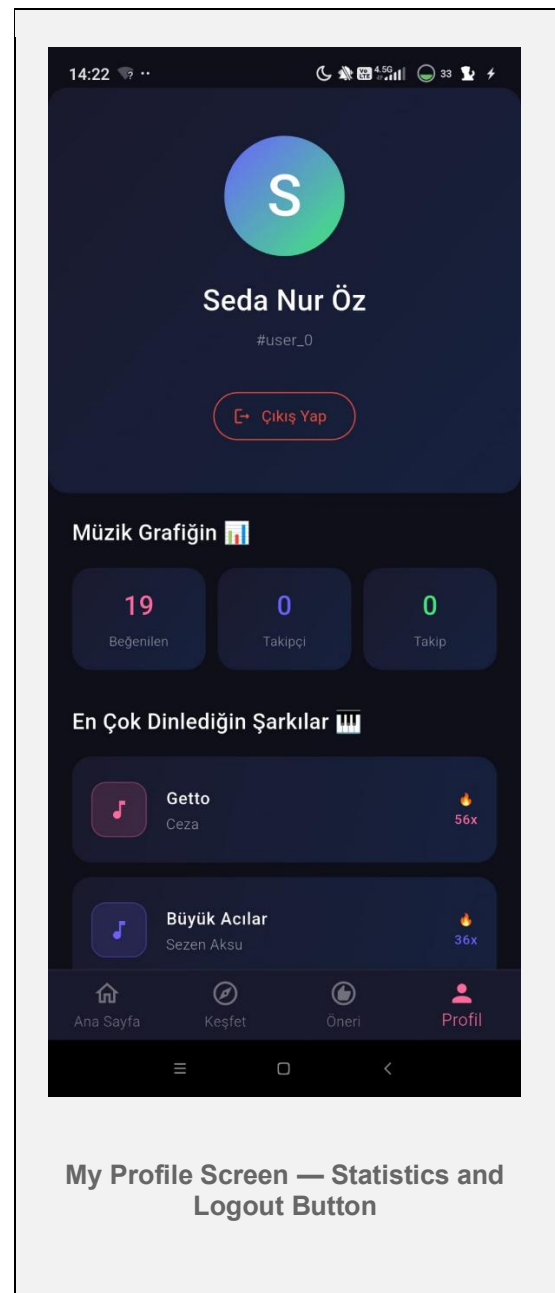
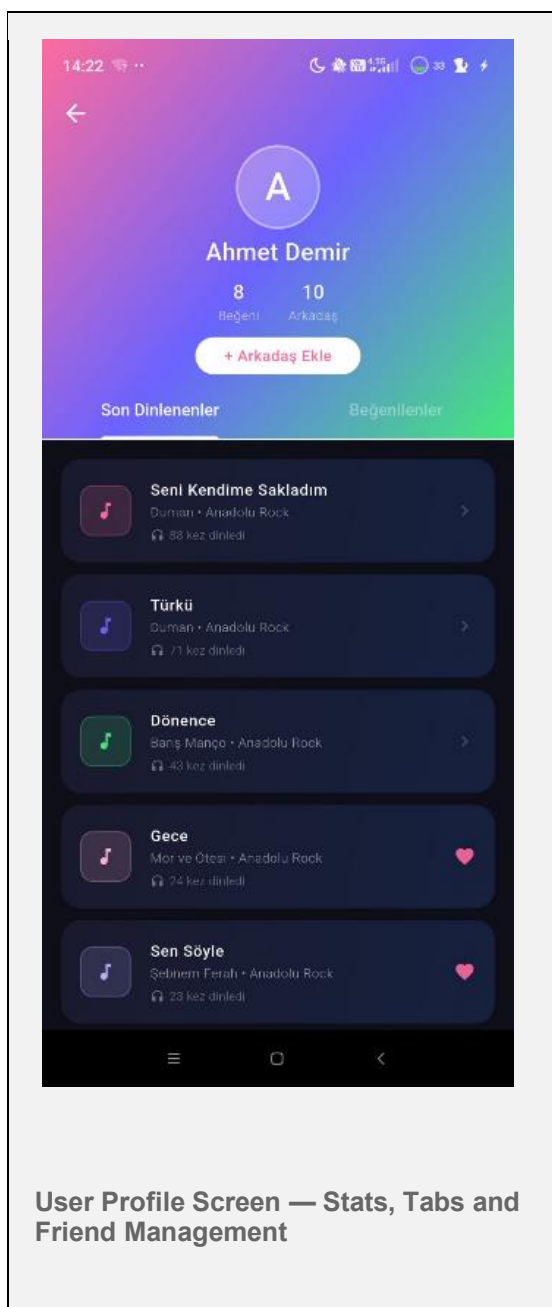
Opened when tapping another user's profile. Content is presented in two tabs:

- Recently Played tab: Songs sorted by the count value of the LISTENED_TO relationship
- Liked Songs tab: Songs connected via the LIKED relationship

The header area shows like count, friend count, and top genre statistics. The Add Friend / Remove Friend button manages the FRIENDS_WITH relationship in Neo4j in real time.

2.7 My Profile Screen (MyProfileScreen)

The screen where the logged-in user views their own profile. Contains listening statistics, a top genre badge, and a logout option.



3. Graph Database Design

3.1 Dataset

The database is populated from two sources:

- **Kaggle Spotify Dataset (tracks_features.csv):** The first 3,000 songs were selected from this dataset of over 1.2 million tracks. Audio features (energy, danceability, tempo, valence) are available for each song, and genre classification was performed based on these values during the Python import process.
- **Manual Turkish Music Data:** 14 artists and 140 songs across Arabesk, Turkish Pop, Turkish Rap, and Anatolian Rock genres were added manually via Cypher queries.

3.2 Node Types and Properties

Node Type	Count	Key Properties
Song	3,190	songId, title, energy, danceability, tempo, valence
Artist	458	artistId, name
Album	270	albumId, title, year
User	30	userId, name, email
Genre	14	name

3.3 Relationship Types

Relationship	Count	Direction	Property	Purpose
PERFORMED	3,676	Artist→Song	—	Artist-song connection
BELONGS_TO	3,190	Song→Genre	—	Genre classification
PART_OF	3,000	Song→Album	—	Song-album link
RELEASED	509	Artist→Album	—	Artist-album link
LISTENED_TO	370	User→Song	count (int)	Play counter, profile stats
LIKED	251	User→Song	—	Favorites, taste matching
FOLLOWS	120	User→Artist	—	Artist follow
FRIENDS_WITH	116	Undirected	—	Social network, recommendation basis

4. Backend API Layer

4.1 API Overview

The backend is built with the Python programming language and the FastAPI framework. It connects to the database via the official Neo4j Python driver (neo4j package). All queries are parameterized — no string concatenation is used.

4.2 Endpoint List

Method	Endpoint	Description
GET	/songs/popular?genre=Pop	Genre-based song list
GET	/songs/search/query?q=...	Song search
GET	/songs/{id}	Song detail
POST	/songs/{id}/like	Like a song (add LIKED)
DELETE	/songs/{id}/like	Unlike a song (remove LIKED)
POST	/songs/{id}/listen	Increment listen counter
GET	/songs/{id}/liked-by-friends	Friends who liked this song
GET	/users/	All users
GET	/users/{id}/profile	User profile statistics
GET	/users/{id}/top-songs	Most-listened songs
GET	/users/{id}/liked-songs	Liked songs
GET	/users/{id}/taste-twins	Taste twins (graph traversal)
GET	/users/{id}/social-radar	Friend-based recommendations
POST	/users/{id}/friend/{target}	Add friend (FRIENDS_WITH)
GET	/artists/{id}/songs	Artist songs

5. Advantages of the Graph Model

5.1 Comparison with the Relational Model

MusicGraph's core recommendation query — Finding Taste Twins — would require multiple JOIN operations in a relational database:

Relational Database (SQL)	Graph Database (Cypher)
<pre>SELECT other.name, COUNT(*) FROM likes l1 JOIN likes l2 ON l1.song_id = l2.song_id JOIN users u ON l2.user_id = u.id WHERE l1.user_id = ? AND l2.user_id != ? GROUP BY other.name ORDER BY COUNT(*) DESC</pre>	<pre>MATCH (u:User {userId: \$uid}) -[:LIKED]->(s:Song) <-[:LIKED]-(other:User) WHERE other.userId <> \$uid RETURN other.name, COUNT(s) AS commonLikes ORDER BY commonLikes DESC</pre>

Because the graph model treats relationships as first-class citizens, it can traverse connections directly without the need for a join table. This significantly reduces both query complexity and the performance degradation that arises in deep relationship queries (such as friend-of-friend-of-friend scenarios).

5.2 Key Benefits of the Graph Model

- **Natural Data Representation:** Social network data is inherently relational; the graph model represents this structure without forcing it into flat tables.
- **Traversal Performance:** Demonstrates clear advantages over JOIN-based approaches in deep relationship queries.
- **Flexible Schema:** New node types or relationship types can be added without breaking existing data.
- **Readable Queries:** Cypher queries express business logic more clearly than chained SQL JOINS.
- **Multi-Hop Recommendations:** Chains such as Friend → Listened Song → Genre → Other Users Who Like That Genre can be resolved in a single query.

6. Conclusion

The MusicGraph project demonstrates concretely how well-suited graph databases are for social and music recommendation scenarios. The data model built on Neo4j AuraDB represents a realistic music social network with 3,190 songs, 458 artists, 270 albums, 30 users, and 14 genre nodes, alongside 8 different relationship types totaling over 8,232 relationships.

The key technical contributions of the project can be summarized as follows:

- Graph traversal-based user matching (Taste Twins algorithm)
- Personalized song recommendations via negative pattern matching (Social Radar)
- Atomic counter updates using MERGE + ON MATCH
- Multi-step Cypher queries utilizing WITH pipelines and aggregation
- Clean REST API design with FastAPI and parameterized Neo4j queries
- Flutter mobile application integration with real-time graph updates

The graph database approach provided a more readable, more flexible, and more naturally suited solution for social recommendation scenarios in this project when compared to a relational model.